

前端开发基础

Web · Vue.js · Node.js

课程	《软件工程与计算 II》
讲次	第 6 讲
主题	前端开发基础：从静态页面到全栈应用
预计时长	约 50 分钟



2004年4月1日 — 互联网的一个愚人节

这一天，Google 推送了一封邮件邀请：

"Google 正在内测一项新邮件服务，提供 1 GB 免费存储空间。"

全球网友的第一反应："这肯定是愚人节玩笑"。

- 那个年代，Hotmail 只给 2 MB，Yahoo Mail 给 4 MB
- 但存储空间还不是最震撼的

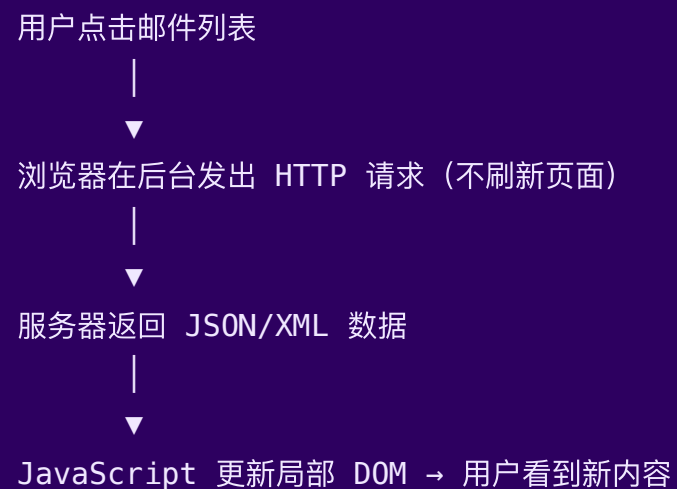
真正让人目瞪口呆的，是用 Gmail 读邮件的感觉：

点击一封邮件 → 内容立刻出现 → 页面没有刷新

这在当时的 Web 世界里，是闻所未闻的。

Gmail 背后的技术链

秘密武器: XMLHttpRequest (XHR)



这一模式被 Jesse James Garrett 在 2005 年命名为 **AJAX**
(Asynchronous JavaScript And XML)

AJAX 引发的前端工程化革命

AJAX 的历史影响链：

Gmail/Google Maps (2004-05) ← AJAX 证明 Web 可以像桌面应用一样
→ jQuery (2006) ← 简化 DOM 操作和 AJAX 调用
→ Backbone / Angular ← 前端 MVC/MV* 框架兴起
→ React (2013) ← 组件化 + 虚拟 DOM
→ Vue.js (2014) ← 渐进式框架, MVVM
→ 现代前端工程化体系 ← Vite / TypeScript / SSR...

Gmail 证明了：Web 页面可以像桌面应用一样流畅响应。
这一刻，"前端开发"作为一门独立工程学科正式诞生。

停下来想一想

在 AJAX 出现之前，每次用户操作都要整页刷新。

假设你在设计一个 2003 年的网页邮箱（无 AJAX）：

- 用户点击“下一页邮件”需要等待整个页面重新加载
- 用户删除一封邮件，页面白屏 1-2 秒后重新显示

问题 1： 整页刷新有哪些用户体验问题？前后端各有什么代价？

问题 2： AJAX 解决了体验问题，但也引入了哪些新的复杂度？

（提示：状态管理、SEO、浏览器前进/后退……）

问题 3： Gmail 用到了 HTML、CSS、JavaScript，你能猜测

它们分别负责邮箱界面的哪些部分吗？

本讲知识框架

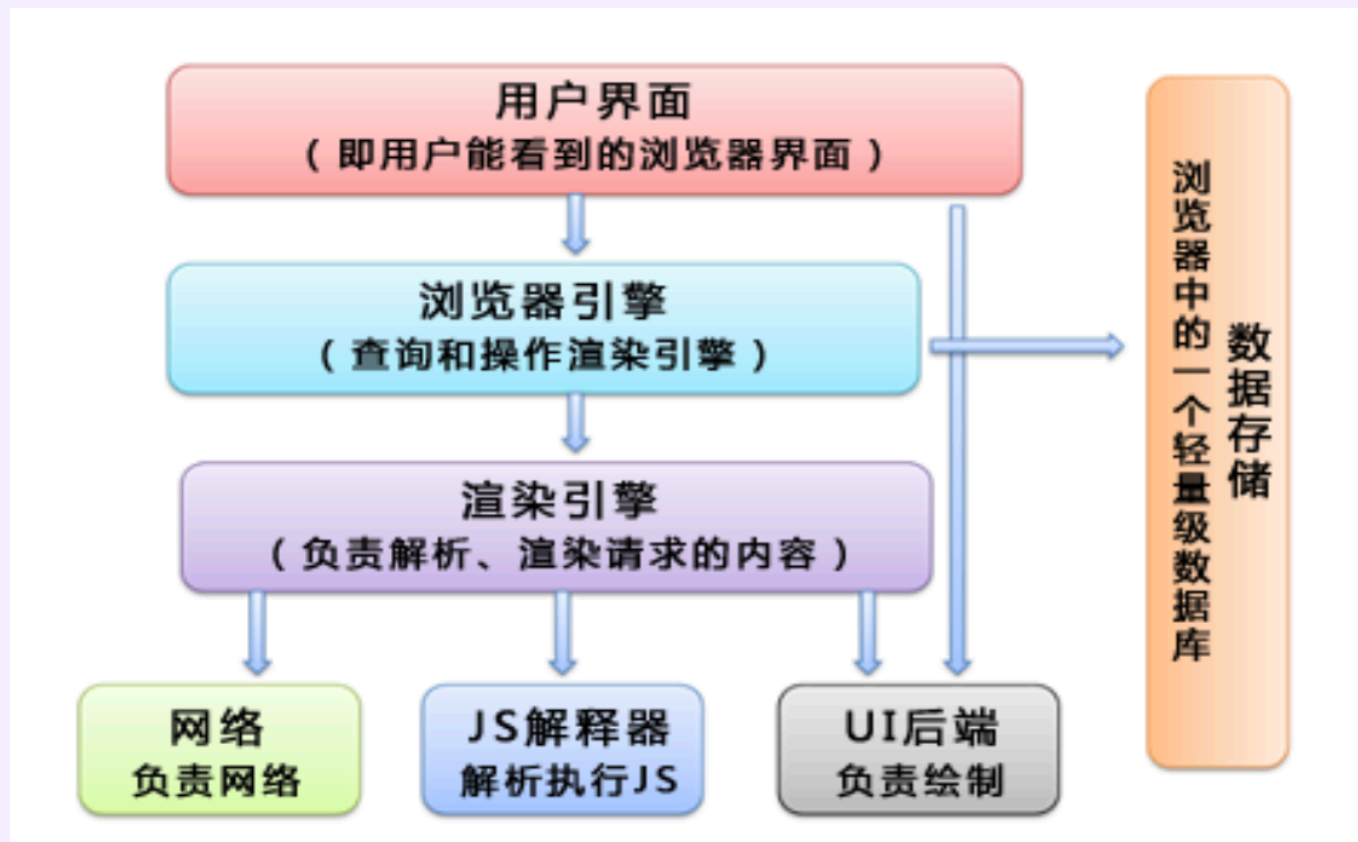


PART 1

Web 前端三层架构

HTML · CSS · JavaScript

浏览器架构：前端技术的运行容器



渲染引擎解析 HTML/CSS 并绘制页面；JS 解释器负责执行 JavaScript。理解这一分工，是掌握前端三层架构（HTML / CSS / JS）的基础。

HTML — 内容与结构层

HyperText Markup Language: 用标签描述内容

```
<!DOCTYPE html>
<html lang="zh">
  <head>
    <meta charset="UTF-8">
    <title>我的网站</title>
  </head>
  <body>
    <h1>网站标题</h1>
    <p>这是一段文字。</p>
    <a href="https://example.com">链接</a>
    
  </body>
</html>
```

HTML 只负责内容和结构，不负责外观和行为。

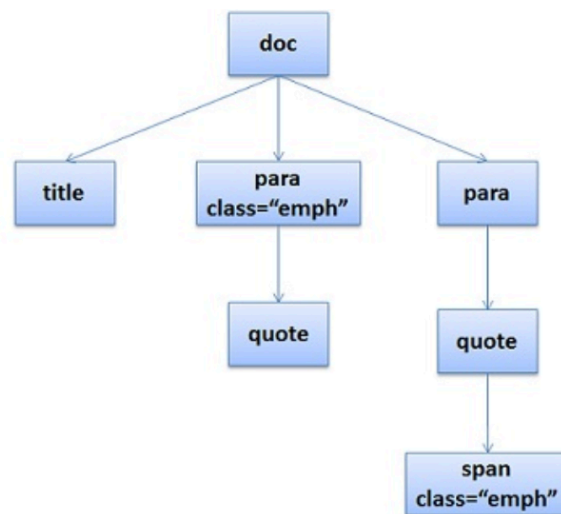
HTML 常用标签速查

标签类别	常用标签	语义
标题	h1 — h6	层级标题
文本	p strong em	段落、强调
媒体	img video audio	嵌入媒体
交互	form input button	用户输入
布局	div section nav	容器/语义块
链接	a	超链接, href 指定目标

语义化 HTML：用正确的标签表达正确的含义（如用 `<nav>` 而非 `<div class="nav">` ），有助于 SEO 和无障碍访问。

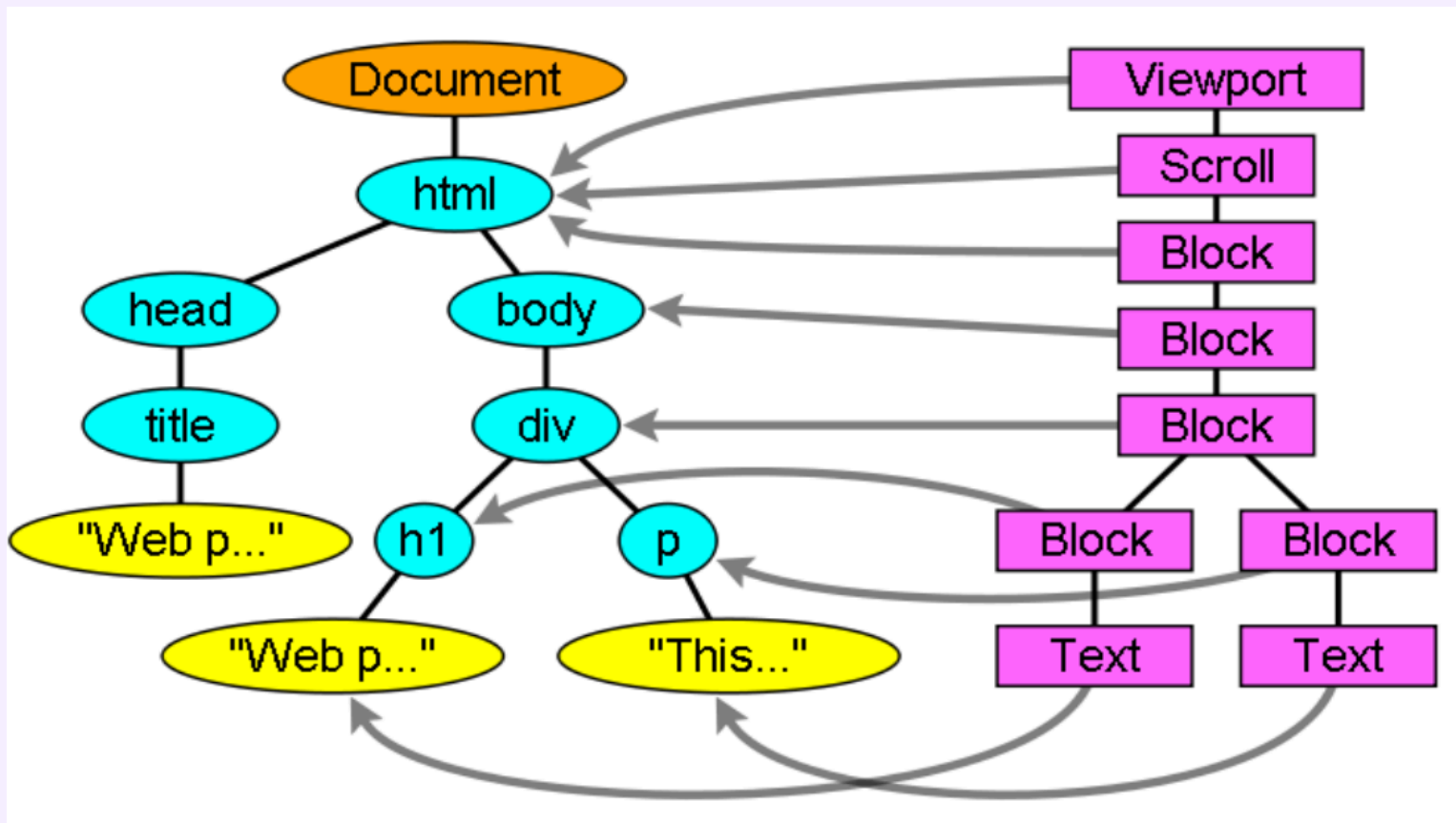
HTML 解析①：标记 → DOM 树

```
<doc>
<title>A few quotes</title>
<para>
  Franklin said that <quote>"A penny saved is a penny earned."</quote>
</para>
<para>
  FDR said <quote>"We have nothing to fear but <span>fear itself.</span>"</quote>
</para>
</doc>
```



浏览器将 HTML 文本解析为内存中的树形结构（DOM），JavaScript 通过 DOM API 读写这棵树。

HTML 解析②：DOM 节点与布局盒



每个 DOM 节点对应一个**布局盒** (Layout Box)，决定其在页面上的位置与尺寸。

CSS — 样式与表现层

Cascading Style Sheets: 控制外观

三种引入方式（优先级由低到高）：

```
/* ① 外部样式表 (推荐) */
/* 在 <head> 中: <link rel="stylesheet" href="style.css"> */

/* ② 内部样式表 */
/* 在 <head> 中: <style> ... </style> */

/* ③ 行内样式 (优先级最高, 不推荐滥用) */
/* <p style="color: red;"> */
```

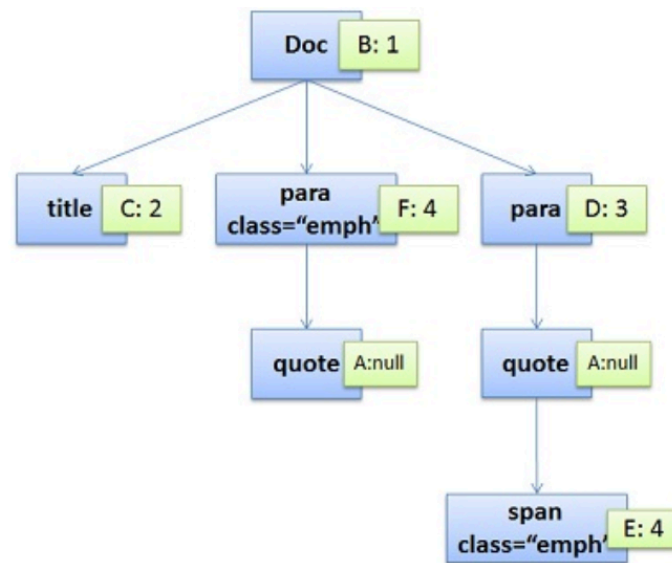
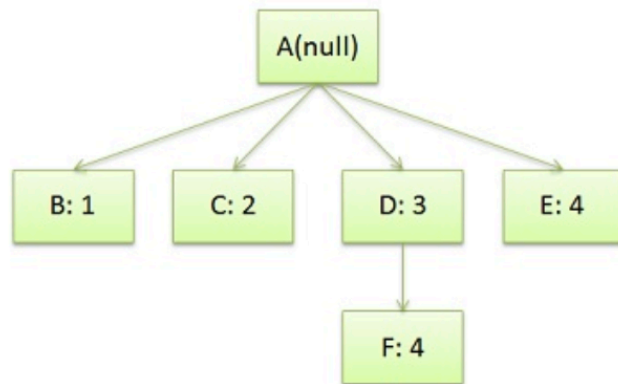
三类选择器：

```
p      { color: #333; }      /* 元素选择器 */
.card  { border-radius: 8px; } /* 类选择器   */
#logo  { width: 120px; }     /* ID 选择器  */
```

CSS 只负责外观和布局，不负责内容和行为。

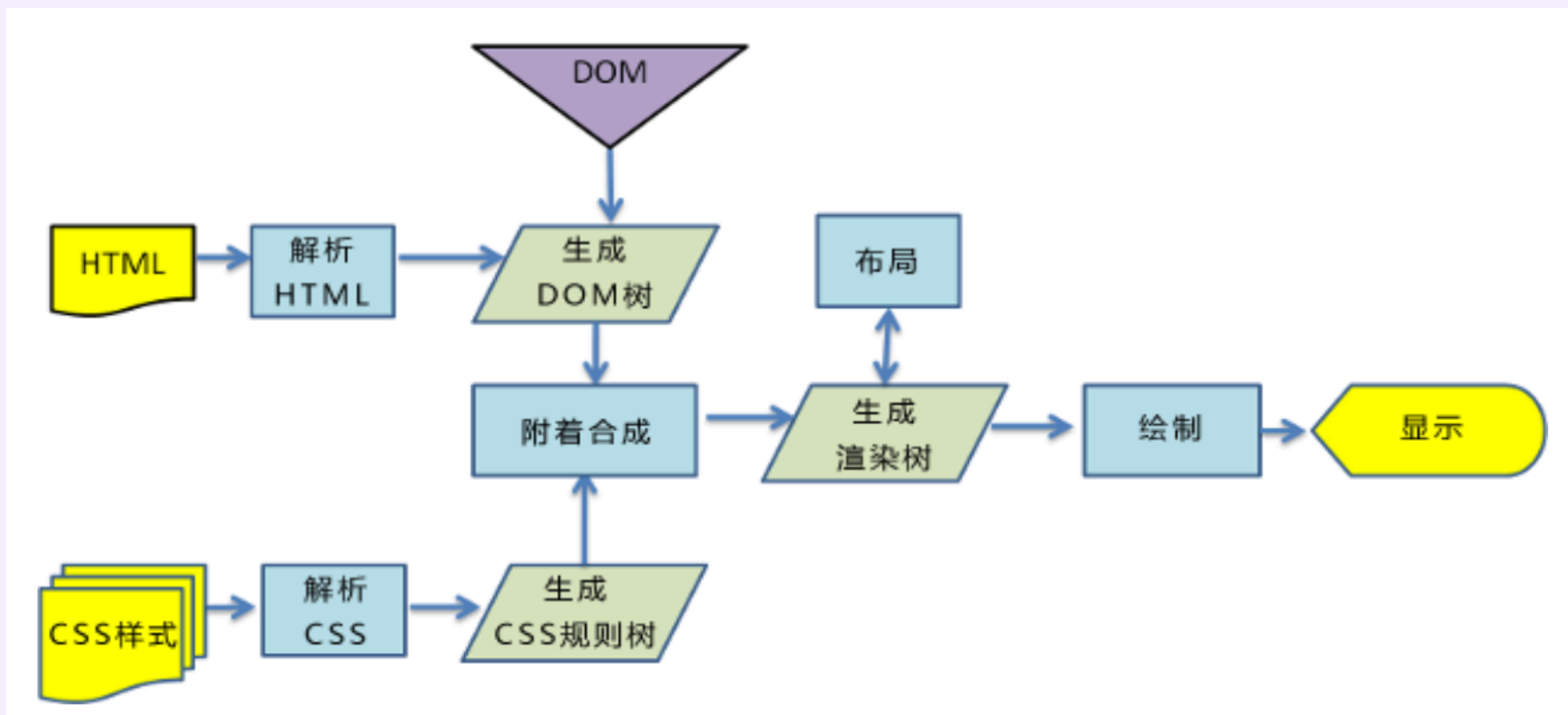
CSS 规则如何附着到 DOM 节点

```
/* rule 1 */ doc { display: block; text-indent: 1em; }  
/* rule 2 */ title { display: block; font-size: 3em; }  
/* rule 3 */ para { display: block; }  
/* rule 4 */ [class="emph"] { font-style: italic; }
```



CSS 引擎将每条规则与 DOM 节点匹配，计算最终样式（Computed Style），形成渲染树（Render Tree）。

浏览器渲染流水线



HTML → DOM 树, CSS → CSS 规则树, 二者合并 → 渲染树 → 布局（位置/大小）→ 绘制 → 显示
JavaScript 可在任意阶段修改 DOM 或样式, 触发重新布局（Reflow）或重绘（Repaint）。

JavaScript — 动态行为层

让页面"活"起来：响应用户操作、异步更新数据

```
// 1. 操作 DOM: 响应点击事件
document.getElementById('btn').addEventListener('click', () => {
  document.getElementById('msg').textContent = '你点击了按钮! ';
});

// 2. 异步请求 (AJAX / Fetch API): 不刷新页面获取数据
fetch('/api/data')
  .then(res => res.json())
  .then(data => {
    console.log(data);
    // 用数据更新页面 DOM
  });
```

JavaScript 只负责交互行为和数据通信，不负责内容结构和外观样式。

JavaScript 四大核心能力

能力	说明	典型用法
DOM 操作	动态增删改查 HTML 元素	<code>getElementById</code> <code>querySelector</code>
事件处理	响应点击、输入、滚动等	<code>addEventListener('click', fn)</code>
异步通信	<code>fetch</code> / XHR 与服务器交互	<code>fetch('/api').then(...)</code>
逻辑计算	变量、函数、条件、循环	ES6+ 语法：箭头函数、解构、模块

为什么这很重要？

AJAX 之后，JS 从"点击弹窗的玩具"变成了构建完整应用的语言，催生了 Node.js（服务端）和现代前端框架（Vue / React）。

三层架构的分工协作

一个网页 = 三层叠加

HTML	—	骨架（内容、结构、语义）
CSS	—	皮肤（颜色、字体、布局、动画）
JS	—	肌肉（交互、数据、动态更新）

类比：

- HTML 是一栋楼的**钢筋混凝土结构**
- CSS 是**室内装修**（颜色、灯光、家具）
- JavaScript 是**电梯和自动门**（响应人的操作）

三层分离的好处：

- 结构清晰，各司其职，便于团队协作
- CSS 换皮不影响内容；JS 逻辑不污染 HTML
- 浏览器可独立缓存各层资源，提升性能

PART 2

Vue.js 与现代前端框架

MVVM · 响应式 · 组件化

为什么需要前端框架？

原生 JS 的痛苦：大量手动 DOM 操作

```
// 数据变了，必须手动找到 DOM 元素并更新  
const count = 0;  
document.getElementById('counter').textContent = count;  
document.getElementById('btn').onclick = () => {  
  count++;  
  document.getElementById('counter').textContent = count; // 手动同步!  
};
```

随着页面复杂度增加，这会演变为噩梦：

- 数据与视图严重耦合，改一处要同步改多处 DOM
- 状态分散在多个变量中，难以追踪
- 代码重复，模块间通信混乱，维护成本爆炸

框架的核心思想：数据驱动视图

你只需要关心数据的变化，框架自动帮你同步更新 DOM。

- ✗

原生 JS：
改数据 → 手动找 DOM → 手动更新 → 漏更新 → Bug
- ✓

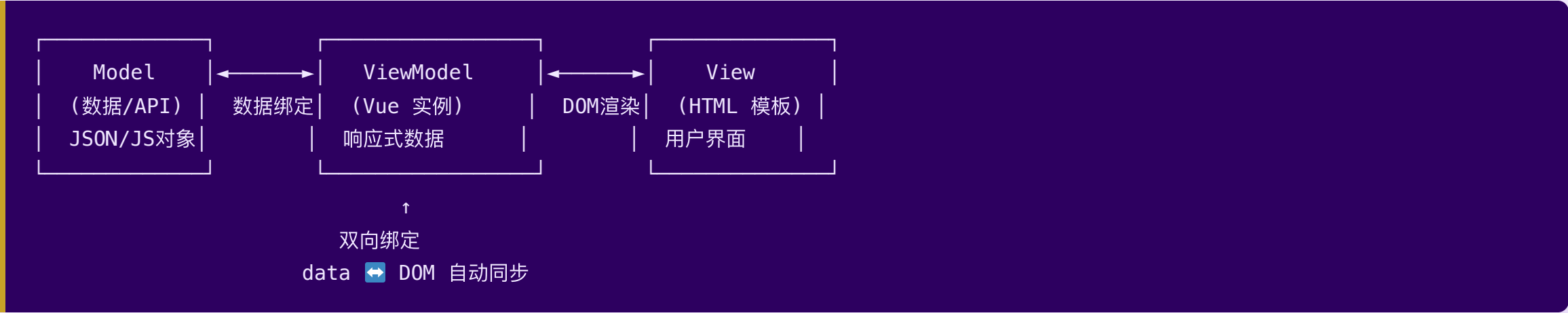
Vue.js：
改数据 → 框架自动更新 DOM → 用户看到新界面

对比一下同一个计数器：

	原生 JS	Vue.js
数据更新	<code>document.getElementById(...).textContent = count</code>	<code>count.value++</code>
视图同步	手动，容易遗漏	自动，声明式
代码量	随功能线性增长	相对稳定

MVVM 与 Vue.js 的响应式原理

MVVM = Model · View · ViewModel



Vue.js 三大核心特性：

特性	说明	效果
响应式数据	数据变化自动触发视图更新	无需手动操作 DOM
组件化	UI 拆分为可复用的独立单元	代码复用、易维护
渐进式	可逐步引入，不强制全量采用	适合大小项目

单文件组件 (SFC)

Vue 的核心文件格式: `.vue` — 模板 · 逻辑 · 样式三合一

```
<!-- MyComponent.vue -->
<template>                                <!-- ① 视图层: HTML 模板 + Vue 指令 -->
  <div class="card">
    <h2>{{ title }}</h2>
    <p>计数: {{ count }}</p>
    <button @click="increment">+1</button>
  </div>
</template>
<script setup>                             <!-- ② 逻辑层: Composition API -->
import { ref } from 'vue'
const title = '我的计数器'
const count = ref(0)
const increment = () => { count.value++ }
</script>
<style scoped>                             /* ③ 样式层: scoped 防止样式泄漏 */
.card { border: 1px solid #ccc; padding: 16px; }
</style>
```

单文件组件（SFC）：为什么这样设计？

传统三文件 vs SFC

维度	传统三文件（HTML / CSS / JS）	SFC（.vue）
文件组织	按技术分层	按组件聚合
复用单元	零散，跨文件复制	组件整体复用，自包含
样式隔离	全局 CSS 互相污染	<code>scoped</code> 只作用本组件
维护成本	改一功能需跨多文件	改动集中在一个文件

组件是前端工程化的基本单元

SFC 让每个组件自包含，大型应用可拆分为数百个独立组件按需复用。

Vue 核心指令速查

指令	用途	示例
<code>v-bind</code> / <code>:</code>	动态绑定属性	<code>:href="url"</code> <code>:class="cls"</code>
<code>v-model</code>	双向数据绑定（表单）	<code><input v-model="name"></code>
<code>v-if</code> / <code>v-else</code>	条件渲染（真正增删DOM）	<code><p v-if="show"></code>
<code>v-show</code>	条件显示（切换 display）	<code><p v-show="show"></code>
<code>v-for</code>	列表渲染	<code><li v-for="item in list" :key="item.id"></code>
<code>v-on</code> / <code>@</code>	事件监听	<code>@click="fn"</code> <code>@input="fn"</code>

`v-if` vs `v-show` 的选择：

- `v-if` — 元素频繁切换少，或初始不渲染 → 适合条件分支
- `v-show` — 元素频繁切换（只改 CSS） → 适合 tab / 折叠面板

记忆口诀： bind绑属性， model双向， if条件， for列表， on事件。

Options API vs Composition API

两种方式实现同一个计数器:

```
<!-- Options API (Vue 2 风格, 按"选项类型"组织) -->
<script>
export default {
  data() { return { count: 0 } },
  methods: { increment() { this.count++ } },
  mounted() { console.log('挂载完成') }
}
</script>
```

```
<!-- Composition API (Vue 3 推荐, 按"功能逻辑"组织) -->
<script setup>
import { ref, onMounted } from 'vue'
const count = ref(0)
const increment = () => { count.value++ }
onMounted(() => { console.log('挂载完成') })
</script>
```

Options API vs Composition API: 如何选择?

维度	Options API	Composition API
逻辑组织	按选项类型分散	按功能聚合
复用方式	Mixins（有冲突风险）	Composables（清晰隔离）
TypeScript	支持有限	完整支持
推荐场景	简单组件 / 迁移项目	新项目 / 复杂逻辑

Vue 3 官方推荐 Composition API + `<script setup>`

新项目直接用 Composition API；阅读老代码会遇到 Options API，两者都需要认识。

动手练习：创建第一个 Vue 项目

 5 分钟，跟着步骤操作

```
# 前提：已安装 Node.js (node -v 验证)

# 1. 用 Vite 创建项目
npm create vite@latest my-app -- --template vue

# 2. 进入项目目录，安装依赖
cd my-app
npm install

# 3. 启动开发服务器
npm run dev

# 浏览器打开 http://localhost:5173
```

修改 `src/App.vue`，把标题改成你的名字：

```
<template>
  <h1>你好, {{ name }}! </h1>
</template>
<script setup>
const name = '张三' // 改成你的名字
</script>
```

PART 3

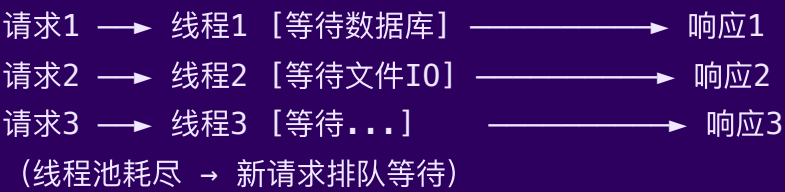
| Node.js 与全栈联动

JavaScript 走向服务端

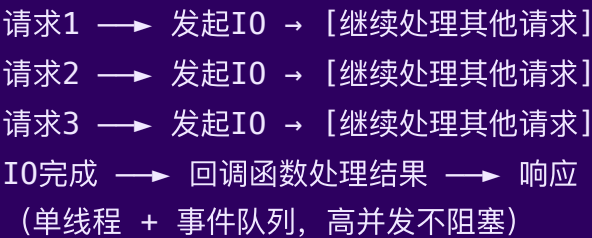
Node.js — JavaScript 的后端力量

Node.js = V8 引擎 + 事件循环 + 非阻塞 I/O

传统服务器（多线程模型）：



Node.js（事件驱动 + 非阻塞）：



核心优势：

特性	说明
高并发	事件驱动，单线程处理大量并发连接
前后同语言	全栈均用 JavaScript，降低切换成本
生态丰富	npm 拥有 200 万+ 包

从 Hello World 到 HTTP 服务器

Node.js Hello World:

```
// hello.js
console.log('Hello, World!');
// 运行: node hello.js
```

构建一个原生 HTTP 服务器:

```
// server.js
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
  res.end('Hello, Node.js!');
});

server.listen(8080, () => {
  console.log('服务器运行在 http://localhost:8080');
});
```

Express 框架：让路由和中间件更简洁

原生 `http` 模块需要手动解析 URL，Express 封装了常用模式：

```
const express = require('express');
const app = express();

// GET 路由
app.get('/api/hello', (req, res) => {
  res.json({ message: 'Hello from Express!' });
});

// 动态路由参数
app.get('/api/user/:id', (req, res) => {
  res.json({ userId: req.params.id });
});

app.listen(3000, () => console.log('Express 服务器已启动'));
```

Express 核心 = 路由 + 中间件

路由决定"谁来处理"，中间件决定"怎么处理"（日志、鉴权、解析 body...）

前后端全栈联动架构

综合案例：网站列表系统（Vue + Node.js + Express）



前后端联动：关键代码

Express 服务端 (server.js):

```
const express = require('express');
const cors    = require('cors');
const sites   = require('./data.json');

const app = express();
app.use(cors()); // 允许跨域 (前端与后端不同端口)
app.get('/getWebSites', (req, res) => res.json(sites));
app.listen(8081, () => console.log('API 服务已启动'));
```

Vue 前端 (WebSites.vue) 调用:

```
import axios from 'axios'
import { ref, onMounted } from 'vue'
const sites = ref([])
onMounted(async () => {
  const { data } = await axios.get('http://localhost:8081/getWebSites')
  sites.value = data
})
```

跨域 (CORS): 浏览器安全策略禁止不同端口的请求,

💬 思考题：路由的两种形态

前端路由（Vue Router）vs 后端路由（Express）

前端路由（SPA 模式）：

- 用户点击 "关于我们"
- URL 变为 /about
- Vue Router 拦截，加载 About.vue 组件
- 页面不刷新，只替换组件
- 服务器实际上没有 /about 这个文件

后端路由（传统模式）：

- 用户点击 "关于我们"
- 浏览器向服务器请求 /about
- 服务器查找路由表，返回对应 HTML 页面
- 整页刷新

请思考并讨论：

维度	前端路由	后端路由
用户体验	✅ 流畅，无白屏	❌ 每次刷新
SEO	❌ 爬虫难抓取	✅ 友好
首屏加载	❌ JS 包较大	✅ 直接返回 HTML

本讲小结

三个核心结论：

① Web 前端 = 三层分工

HTML 定义内容与结构，CSS 控制外观与布局，JavaScript 实现交互与数据更新。三层各司其职，分离关注点，是前端工程化的基础。

② Vue.js 以数据驱动视图

MVVM 模式将开发者从手动 DOM 操作中解放出来。单文件组件（SFC）封装模板、逻辑、样式；Composition API 以功能聚合替代选项分散，更易复用和维护。

③ Node.js 打通前后端

同一语言（JavaScript）横跨浏览器和服务器，降低了全栈开发门槛。事件驱动的非阻塞 I/O 使 Node.js 擅长高并发的 API 服务和实时场景。

一条技术演进线索：

静态 HTML → 动态 JS → AJAX 异步 → 前端框架 → 组件化 → 全栈 JS

每一步演进都是为了解决上一步遗留的工程复杂性问题。

课后作业

本讲作业（4 项，二选二完成）

A. 基础巩固

1. 用原生 HTML + CSS + JS 实现一个"待办事项"页面：支持添加/删除条目，无需框架，纯手动 DOM 操作。体会框架出现前的开发方式。
2. 阅读 Vue.js 官方文档"快速上手"章节（中文版），对照本讲内容，找出至少 3 处本讲未覆盖的知识点，写成笔记。

B. 动手实践

3. 在课堂 Vue 计数器基础上，增加"减法"按钮和"重置"按钮，并用 `v-show` 在计数为 0 时显示"归零啦"提示。
4. 参考课堂网站列表案例，将 `data.json` 扩展为至少 5 条数据，为每条数据增加 `category`（分类）字段，并在前端用 `v-for` + `v-if` 实现按分类过滤展示。

提交格式： 代码截图 + 运行效果截图，下次课前发至课程群。

下课前，三分钟回顾

今天学了什么？

Web 前端三层架构

- └─ HTML — 骨架：标签、语义、结构
- └─ CSS — 皮肤：选择器、样式、布局
- └─ JS — 动作：DOM操作、事件、Fetch

Vue.js

- └─ MVVM：数据驱动视图，告别手动 DOM
- └─ SFC (.vue)：template + script + style 三合一
- └─ 核心指令：v-bind / v-model / v-if / v-for / v-on
- └─ Composition API：<script setup> + ref + 生命周期钩子

Node.js

- └─ V8 引擎 + 事件循环 + 非阻塞 I/O
- └─ 内置 http 模块 → 构建服务器
- └─ Express 框架 → 路由 + 中间件
- └─ 前后端联动：Vue(5173) ↔ axios ↔ Node.js(8081)

下节预告

第 5 讲：Java Web 开发

1.47 亿条个人信息，在 78 天内被悄悄拿走——它揭示了后端安全的致命代价，也是我们学 Spring Boot 的起点.....

南京大学 · 《软件工程与计算 II》 · 2026 春季学期

第 4 讲 前端开发基础

本讲核心收获

- Web 演进：静态页面 → AJAX → SPA，理解现代 Web 的底层逻辑
- Vue.js：声明式渲染 + 响应式数据，SFC (template/script/style) 三合一
- Node.js：V8 + 事件循环 + Express，前后端统一使用 JavaScript

HTML · CSS · JavaScript · Vue.js · Node.js